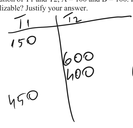
II. Solve the following problems:

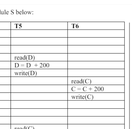
1. Consider the schedule below: (0.5p)

T1	T2
read(A)	
A = A - 50	
write(A)	
	read(A)
	A = A * 4
	write(A)
	read(B)
	B = B * 4
	write(B)
read(B)	
B = B + 50	
write(B)	

time

The following integrity constraint must hold: $A = B$. Before the execution of T1 and T2, A = 100 and B = 100. Is the schedule serializable? Justify your answer.





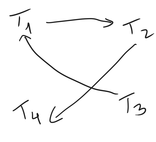
1. Consider schedule S below:

T4	T5	T6
read(C)		
C = C + 100		
write(C)		
	read(D)	
	D = D + 200	
	write(D)	
		read(C)
		C = C + 200
		write(C)
read(D)		
D = D + 100		
write(D)		
	read(C)	
	C = C + 100	
	write(C)	
		read(D)
		D = D + 100
		write(D)

time

Before the execution above, C = 100 and D = 100.

a. S is not serializable.
b. S is serializable. A serial schedule that's equivalent to S (in its effect on the database) is: T4 followed by T5 followed by T6.
c. S is serializable. A serial schedule that's equivalent to S (in its effect on the database) is: T4 followed by T6 followed by T5.
d. S is serializable. A serial schedule that's equivalent to S (in its effect on the database) is: T5 followed by T4 followed by T6.
e. None of the above answers is correct.



2. Consider the following schedule over transactions T1, T2, T3, T4 (all transactions commit): (1p) 4p

T1	T2	T3	T4
W(A)			R(C)
	R(B)		
		W(D)	
	R(A)		
R(D)			
			W(B)
R(C)			

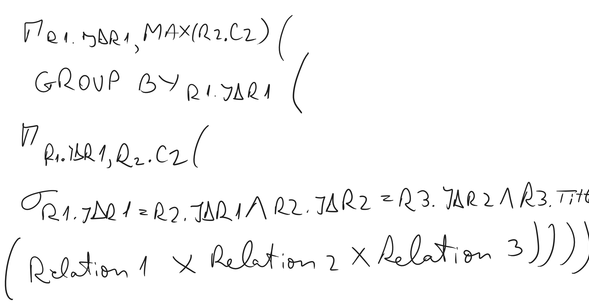
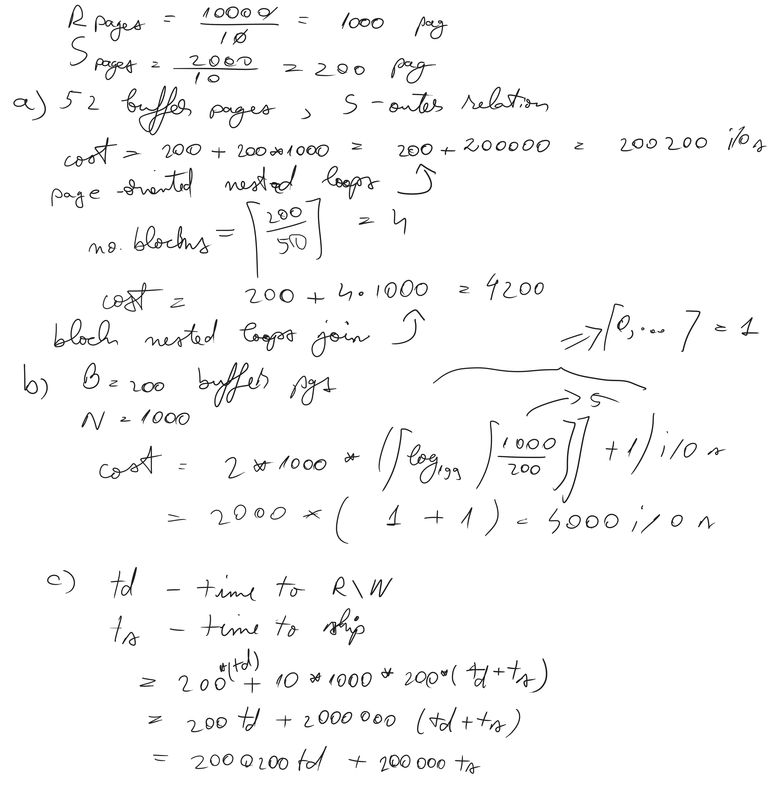
time

a. Draw the precedence graph.
b. Is the schedule conflict serializable? Justify your answer.

The precedence graph is an acyclic graph therefore is conflict serializable.

3. Express the SQL query below in the extended relational algebra, using a σ π $\times$ expression.

```
SELECT R1.IDR1, MAX(R2.C2)
FROM Relation1 R1, Relation2 R2, Relation3 R3
WHERE R1.IDR1 = R2.IDR1 AND R2.IDR2 = R3.IDR2 AND R3.Title = 'TBA' AND R1.C1 = 20
GROUP BY R1.IDR1
```



4. Let R and S be 2 relations. R has 10.000 records; a page can hold 10 R records. S has 2.000 records; a page can hold 10 S records. (1.5p)
a. 52 buffer pages are available. Compute the cost of $R \otimes_{R.a=S.b} S$ using *page-oriented nested loops join* and *block nested loops join*; S is the outer relation.
b. Compute the cost of sorting R using *external merge sort* with 200 buffer pages.
c. R is stored at Madrid, S is stored at Athens. Compute the cost of $R \otimes_{R.a=S.b} S$ using *simple nested loops join (tuple-oriented)* in Athens, without caching; S is the outer relation.